DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining

Assignment 3 – CO3

Course Outcome: CO3 – Understand the functionality of the various data mining components and expose them to real-world data sets to analyse their behaviour.

CO3 Coverage: *CLASSIFICATION AND PREDICTION: Issues regarding classification and prediction, decision tree induction, Bayesian classification, rule-based classification, backpropagation, support vector machines, associative classification, accuracy and error measures, evaluating a classifier or predictor (cross-validation and bootstrap), metrics for performance evaluation, prediction algorithms, and the Weka tool for mining.*

Mode: Pair Work – **2 Students per Group** | **Duration:** 1 Week | **Total Marks:** 50

Pair-Work Principle

Both students are responsible for understanding the **complete assignment**. However, each question contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must not divide the assignment question-wise. Both students must participate in every question.

The individual responsibilities intentionally alternate across questions so that both students demonstrate evidence of:

Calculate → Implement → Analyse → Evaluate

ANSWERS

Q1. DECISION TREE INDUCTION: CHOOSING THE SPLITTING ATTRIBUTE:

1. Class attribute, candidate attributes and stopping conditions

The class attribute is Approved, which has two possible values: Yes and No.

The candidate splitting attributes are:

1. Income
2. Credit History
3. Employment

The decision tree stops when all records belong to the same class, no attributes remain, or no further useful information gain is obtained.

2. Entropy of the complete dataset

Total records = 12

Approved = Yes: 5

Approved = No: 7

Formula:

$$Entropy(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

$$= -\frac{5}{12}\log_2 \frac{5}{12} - \frac{7}{12}\log_2 \frac{7}{12}$$

$$\boxed{Entropy(S) = 0.9799}$$

3. Information Gain of each attribute

Formula:

$$IG(A) = Entropy(S) - Entropy(S|A)$$

Income:

- High: 3 Yes, 1 No
- Medium: 1 Yes, 3 No
- Low: 1 Yes, 3 No

Weighted entropy = 0.8742

$$IG(Income) = 0.9799 - 0.8742$$

$$\boxed{IG(Income) = 0.1057}$$

Credit History:

- Good: 4 Yes, 2 No
- Poor: 1 Yes, 5 No

Weighted entropy = 0.8091

$$IG(Credit) = 0.9799 - 0.8091$$

$$\boxed{IG(Credit) = 0.1708}$$

Employment:

- Salaried: 3 Yes, 3 No
- Self-employed: 2 Yes, 4 No

Weighted entropy = 0.9591

$$IG(Employment) = 0.9799 - 0.9591$$

$$\boxed{IG(Employment) = 0.0208}$$

4. Choosing the root attribute

Credit History has the highest information gain of 0.1708. Therefore, it is selected as the root attribute.

5. Gain Ratio and Gini Index

Gain Ratio

$$GR(A) = \frac{Information\ Gain}{Split\ Information}$$

The gain ratios are:

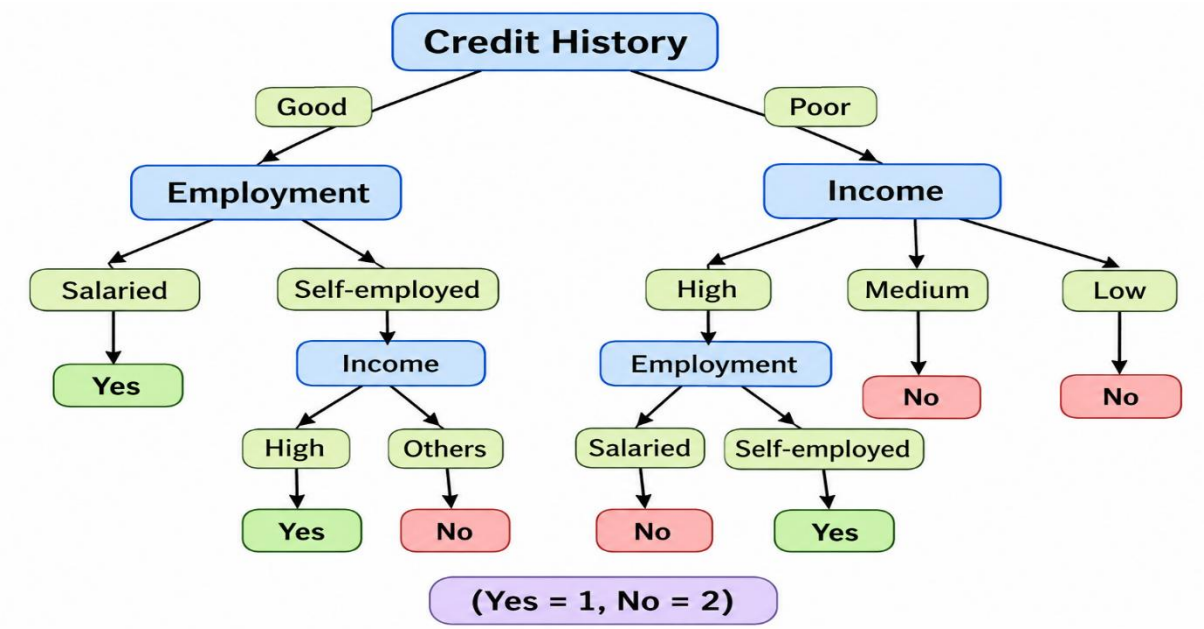
- Income = 0.1064
- Credit History = 0.1957
- Employment = 0.0207

Gini index gain:

- Income = 0.1111
- Credit History = 0.1250
- Employment = 0.0139

Both methods select Credit History as the root attribute. Therefore, the root selection does not change.

6. Complete decision tree



7. Decision tree rules

1. IF Credit History = Good AND Employment = Salaried, THEN Approved = Yes.
2. IF Credit History = Good AND Employment = Self-employed AND Income = High, THEN Approved = Yes.
3. IF Credit History = Good AND Employment = Self-employed AND Income = Medium or Low, THEN Approved = No.
4. IF Credit History = Poor AND Income = Medium or Low, THEN Approved = No.
5. IF Credit History = Poor AND Income = High AND Employment = Salaried, THEN Approved = No.
6. IF Credit History = Poor AND Income = High AND Employment = Self-employed, THEN Approved = Yes.

8. Attributes with many distinct values

An attribute such as Customer ID may have a unique value for every record. A decision tree may use it to create separate branches for individual records. This causes overfitting because the model memorizes training data instead of learning general patterns.

9. Weka validation and overfitting

J48 is the decision tree algorithm available in Weka. It can be tested using the 12 loan records and the German Credit (credit-g) dataset.

The pruning confidence factor controls the amount of pruning. A smaller value generally produces a simpler tree. Minimum instances per leaf controls the minimum number of records required in each leaf.

Overfitting occurs when a tree performs very well on training data but poorly on unseen data. Pruning reduces unnecessary branches and improves generalization.

Note: Actual Weka tree size and accuracy must be obtained by running the experiments; they cannot be determined from the assignment data alone.

Q2. BAYESIAN CLASSIFICATION AND THE COST OF ASSUMPTIONS:

1. Given applicant

Income = Low

Credit History = Good

Employment = Self-employed

We use the Naïve Bayes classifier to predict whether the loan will be approved.

2. Prior probabilities

Total records = 12

Yes = 5, No = 7

$$P(Yes) = \frac{5}{12} = 0.4167$$

$$P(No) = \frac{7}{12} = 0.5833$$

3. Conditional probabilities

For Approved = Yes:

$$P(Low|Yes) = \frac{1}{5} = 0.2$$

$$P(Good|Yes) = \frac{4}{5} = 0.8$$

$$P(Self - employed|Yes) = \frac{2}{5} = 0.4$$

For Approved = No:

$$P(Low|No) = \frac{3}{7} = 0.4286$$

$$P(Good|No) = \frac{2}{7} = 0.2857$$

$$P(Self - employed|No) = \frac{4}{7} = 0.5714$$

4. Posterior probability calculation

For Yes:

$$P(Yes|X) \propto 0.4167 \times 0.2 \times 0.8 \times 0.4$$
$$= 0.02667$$

For No:

$$P(No|X) \propto 0.5833 \times 0.4286 \times 0.2857 \times 0.5714$$
$$= 0.04082$$

Since 0.04082 is greater than 0.02667, the predicted class is:

Approved = No

5. Laplace correction

Laplace correction is used to avoid zero probabilities in Naïve Bayes.

Formula:

$$P(A|C) = \frac{Count(A, C) + 1}{Count(C) + k}$$

Here, k is the number of possible attribute values.

For Yes:

- Income Low = 2/7
- Credit Good = 5/7
- Employment Self-employed = 3/7

For No:

- Income Low = 4/9
- Credit Good = 3/9
- Employment Self-employed = 5/9

Recalculated posterior values:

$$P(Yes|X) \propto 0.03644$$

$$P(No|X) \propto 0.04801$$

Therefore, the predicted class remains Approved = No.

6. Weka verification

The NaiveBayes classifier in Weka can be used to verify the calculated probabilities. Load the loan dataset, select NaiveBayes and run the classifier. Compare its class probabilities with the manual calculation.

7. Correlated attributes

Naïve Bayes assumes that all attributes are independent of each other. However, real-world attributes may be strongly correlated.

For example, in a medical dataset, weight and BMI are related attributes. Including both may affect the probability calculations.

A decision tree can handle such relationships by selecting useful attributes at different branches. Naïve Bayes may lose accuracy when the independence assumption is strongly violated.

8. Why Naïve Bayes is still useful

Naïve Bayes can provide useful class rankings even when attributes are correlated because it combines probability estimates from multiple attributes. It is also simple, fast and effective for many classification problems.

9. Joint conclusion

Naïve Bayes is preferred for large datasets, text classification and applications requiring fast prediction. Decision trees are preferred when the relationships between attributes are complex and explanations are important.

Q3. RULE-BASED AND ASSOCIATIVE CLASSIFICATION:

1. Conversion of decision tree into IF–THEN rules

Rules obtained from the decision tree:

1. IF Credit History = Good AND Employment = Salaried, THEN Approved = Yes.
2. IF Credit History = Good AND Employment = Self-employed AND Income = High, THEN Approved = Yes.
3. IF Credit History = Good AND Employment = Self-employed AND Income = Medium or Low, THEN Approved = No.
4. IF Credit History = Poor AND Income = Medium or Low, THEN Approved = No.
5. IF Credit History = Poor AND Income = High AND Employment = Salaried, THEN Approved = No.
6. IF Credit History = Poor AND Income = High AND Employment = Self-employed, THEN Approved = Yes.

2. Coverage and accuracy

Coverage measures how many records are matched by a rule.

Accuracy measures how many matched records are correctly classified.

Formula:

$$Coverage = \frac{Matched\ Records}{12}$$

$$Accuracy = \frac{Correct\ Predictions}{Matched\ Records}$$

For the six rules:

- Rule 1: Coverage = 3/12; Accuracy = 100%.
- Rule 2: Coverage = 1/12; Accuracy = 100%.
- Rule 3: Coverage = 2/12; Accuracy = 100%.
- Rule 4: Coverage = 4/12; Accuracy = 100%.
- Rule 5: Coverage = 1/12; Accuracy = 100%.
- Rule 6: Coverage = 1/12; Accuracy = 100%.

3. Rule conflict resolution

Rule conflict occurs when two or more rules match the same record but predict different classes.

It can be resolved using rule ordering, where higher-priority rules are checked first, or size ordering, where more specific rules are preferred.

For example, a rule containing three conditions can be given priority over a general rule containing only one condition.

4. Sequential covering

Sequential covering is a rule-learning technique that learns one rule at a time. It selects a rule that correctly classifies some training records and removes the covered positive records. The process continues until the required classification is achieved.

5. Associative classification

Associative classification uses class-association rules to connect attribute values with class labels.

Support measures how frequently a rule occurs in the dataset.

Confidence measures how often the rule correctly predicts its class.

$$Support = \frac{Matching\ Records}{Total\ Records}$$

$$Confidence = \frac{Correct\ Predictions}{Matching\ Records}$$

6. Class-association rules with minimum support 25% and confidence 70%

The following rules predict Approved = Yes:

- Credit History = Good AND Employment = Salaried → Approved = Yes.
- Income = High AND Employment = Salaried → Approved = Yes.
- Income = High AND Credit History = Good → Approved = Yes.

Each of these rules has sufficient support and confidence in the given dataset.

7. Weka JRip/PART comparison

JRip and PART are rule-based classification algorithms available in Weka. They generate IF–THEN rules from training data. J48 produces a decision tree, while JRip and PART produce rule-based models.

Actual rule output and accuracy should be recorded after running the German Credit dataset.

8. Joint conclusion

For a loan officer, rule-based classification is a suitable choice because every decision can be explained using simple IF–THEN statements. Decision trees also provide understandable paths, while associative rules identify frequent relationships between attributes and loan decisions.

Q4. BACKPROPAGATION: WHAT DOES THE NETWORK LEARN

1. Given values

- Inputs: x1 = 1, x2 = 0
- Target = 1
- Learning rate = 0.9
- Hidden units = 2
- Output units = 1
- Activation function = Sigmoid

Sigmoid formula:

$$f(x) = \frac{1}{1 + e^{-x}}$$

2. Forward propagation

For hidden unit H1:

$$Net(H1) = (1 \times 0.2) + (0 \times 0.4) - 0.4$$

$$Net(H1) = -0.2$$

$$H1 = f(-0.2) = 0.4502$$

For hidden unit H2:

$$Net(H2) = (1 \times -0.3) + (0 \times 0.1) + 0.2$$

$$Net(H2) = -0.1$$

$$H2 = f(-0.1) = 0.4750$$

For output unit:

$$Net(O) = (0.4502 \times -0.3) + (0.4750 \times -0.2) + 0.1$$

$$Net(O) = -0.1301$$

$$O = f(-0.1301) = 0.4675$$

3. Error calculation

Formula:

$$E = \frac{1}{2}(Target - Output)^2$$

$$E = \frac{1}{2}(1 - 0.4675)^2$$

$$\boxed{E = 0.1418}$$

4. Error term calculation

Output error:

$$\delta_o = (t - o)o(1 - o)$$

$$\delta_o = (1 - 0.4675)(0.4675)(0.5325)$$

$$\delta_o = 0.13256$$

Hidden unit errors:

$$\delta_{h1} = h1(1 - h1)\delta_o(-0.3)$$

$$\delta_{h1} = -0.00984$$

$$\delta_{h2} = h2(1 - h2)\delta_o(-0.2)$$

$$\delta_{h2} = -0.00661$$

5. Weight and bias updates

Formula:

$$New\ Weight = Old\ Weight + \eta\delta x$$

Updated weights:

- x1 to H1 = 0.19114
- x2 to H1 = 0.40000
- x1 to H2 = -0.30595
- x2 to H2 = 0.10000

- H1 to Output = -0.24630
- H2 to Output = -0.14333

Updated biases:

- H1 bias = -0.40886
- H2 bias = 0.19405
- Output bias = 0.21930

6. Recomputed output

After updating the weights and biases:

$$H1 = 0.44578$$

$$H2 = 0.47205$$

$$Output = 0.51046$$

New error:

$$E = \frac{12}{(1 - 0.51046)}^{2}$$

$$\boxed{E = 0.11982}$$

The error decreased from 0.1418 to 0.1198. Therefore, backpropagation successfully improved the network output.

7. Weight initialization

Weights are initialized to small random values to prevent hidden neurons from learning identical features. If all weights have the same value, neurons may learn the same information, reducing the effectiveness of the network.

8. Weka experiment

A MultilayerPerceptron can be tested using Weka. Different learning rates, hidden units and training epochs can be selected to compare training and testing accuracy.

Overfitting occurs when training accuracy increases but testing accuracy decreases. Slow training occurs when the learning rate is too small, while an excessively high learning rate may produce unstable training.

9. Comparison with decision trees

Neural networks can learn complex relationships and nonlinear patterns, but they require more training time and are difficult to interpret. Decision trees are easier to understand and explain, but may be less effective for some complex datasets.

Q5. SUPPORT VECTOR MACHINES: MARGIN, KERNELS AND SCALING

1. Given dataset

Class +1:

- (3,3)
- (4,3)
- (4,4)

Class -1:

- (1,1)
- (1,2)
- (2,1)

A Support Vector Machine (SVM) is a supervised learning algorithm that finds the best separating hyperplane between two classes.

2. Support vectors

The closest points between the two classes are:

- Positive support vector: (3,3)
- Negative support vectors: (2,1) and (1,2)

These points determine the maximum-margin boundary.

3. Separating hyperplane

The separating hyperplane is:

$$w_1 x_1 + w_2 x_2 + b = 0$$

For the given points:

$$\boxed{2x_1 + 2x_2 - 9 = 0}$$

Or

$$\boxed{x_1 + x_2 = 4.5}$$

Points above the boundary belong to Class +1, while points below belong to Class -1.

4. Margin width

The margin width is calculated using:

$$Margin = \frac{2}{\sqrt{w_1^2 + w_2^2}}$$

Using the normalized weights $w_1 = w_2 = \frac{2}{3}$:

$$Margin = \frac{2}{\sqrt{(2/3)^2 + (2/3)^2}}$$

$$\boxed{Margin = 2.1213}$$

5. Removing support vectors

If a non-support-vector point is removed, the separating hyperplane usually remains unchanged. If a support vector is removed, the position of the hyperplane and margin may change.

6. Soft margin and kernels

The soft margin parameter C controls the trade-off between maximizing the margin and minimizing classification errors. A high C penalizes misclassification more strongly, while a low C allows more errors.

Kernels help SVM classify nonlinear data by mapping it into a higher-dimensional feature space. Common kernels include Linear, Polynomial and RBF.

7. Weka experiments

The SMO algorithm in Weka can be used to test SVM classification. Different kernels and values of C can be compared using accuracy and the number of support vectors.

- Linear kernel: Suitable for linearly separable data.
- Polynomial kernel: Suitable for curved relationships.
- RBF kernel: Suitable for complex nonlinear data.

8. Attribute scaling

Scaling converts attributes into a similar numerical range. Without scaling, attributes with larger numerical values may dominate the model. Standardization or normalization helps improve SVM performance.

9. Joint conclusion

SVM is suitable for high-dimensional datasets and classification problems with clear margins. Its limitations include computational cost on large datasets, sensitivity to parameter selection and difficulty in interpreting the model.

Q6. EVALUATING CLASSIFIERS AND PREDICTORS

1. Given values

Total transactions = 1000

- True Positive (TP) = 40
- False Negative (FN) = 20
- False Positive (FP) = 30
- True Negative (TN) = 910

2. Accuracy

Accuracy measures the percentage of correctly classified records.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100$$

$$= \frac{40 + 910}{1000} \times 100$$

$$\boxed{Accuracy = 95\%}$$

3. Error rate

$$Error\ Rate = \frac{FP + FN}{Total} \times 100$$

$$= \frac{30 + 20}{1000} \times 100$$

$$\boxed{Error\ Rate = 5\%}$$

4. Sensitivity or Recall

$$Recall = \frac{TP}{TP + FN}$$

$$= \frac{40}{40 + 20}$$

$$\boxed{Recall = 66.67\%}$$

5. Specificity

$$Specificity = \frac{TN}{TN + FP}$$

$$= \frac{910}{910 + 30}$$

$$\boxed{Specificity = 96.81\%}$$

6. Precision

$$Precision = \frac{TP}{TP + FP}$$

$$= \frac{40}{40 + 30}$$

$$\boxed{Precision = 57.14\%}$$

7. F-measure

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$\boxed{F1 = 61.54\%}$$

8. Class imbalance problem

If a classifier predicts every transaction as not fraud, it correctly classifies 940 transactions.

$$Accuracy = \frac{940}{1000} \times 100 = 94\%$$

Although the accuracy is high, the classifier fails to identify every fraudulent transaction. Therefore, accuracy alone is misleading for imbalanced datasets.

9. Prediction error measures

Actual values = 25

Predicted values = 30

Absolute errors = 5

MAE:

$$MAE = \frac{2 + 3 + 1 + 5}{4}$$

$$\boxed{MAE = 2.75}$$

MSE:

$$MSE = \frac{4 + 9 + 1 + 25}{4}$$

$$\boxed{MSE = 9.75}$$

RMSE

$$RMSE = \sqrt{9.75}$$

$$\boxed{RMSE = 3.12}$$

RMSE is preferred when larger prediction errors must receive greater importance because it squares the errors before calculating the average.

10. ROC curve

A ROC curve represents the relationship between True Positive Rate and False Positive Rate at different classification thresholds.

The X-axis represents False Positive Rate, and the Y-axis represents True Positive Rate. A curve closer to the top-left corner indicates better classification performance.

11. Estimation methods

- Holdout: Divides the dataset into training and testing sets.
- 10-fold cross-validation: Divides the dataset into ten parts and tests the model ten times using different training and testing portions.
- Bootstrap (.632): Uses repeated sampling with replacement to estimate model performance.

12. Classifier comparison

J48, Naïve Bayes and SMO can be compared using Weka Experimenter.

J48 is easy to interpret, Naïve Bayes is computationally efficient, and SMO is useful for high-dimensional classification.

Paired evaluation can determine whether performance differences between classifiers are statistically meaningful.

Note: Actual accuracy estimates and ROC plots require running the experiments on the selected dataset.

Q7. DESIGN A CLASSIFICATION SOLUTION FROM FIRST PRINCIPLES

1. Problem identification

The hospital wants to predict whether a patient will be readmitted within 30 days after discharge.

Only 6% of patients are readmitted. Missing a high-risk patient is ten times more costly than incorrectly flagging a low-risk patient.

The system must provide understandable alerts to doctors.

2. Student A – Initial design proposal

I recommend using a Decision Tree classifier because it provides understandable IF–THEN rules. It can identify important factors such as age, diagnosis, previous admissions and length of stay.

Missing values can be handled using imputation, and class imbalance can be handled using oversampling or cost-sensitive learning.

3. Student B – Initial design proposal

I recommend using Logistic Regression with cost-sensitive learning. It can estimate the probability of patient readmission and adjust the classification threshold based on the cost of errors.

Missing values can be replaced using mean, median or mode imputation.

4. Comparison of both designs

Decision Trees are easy to understand and provide clear decision rules. Logistic Regression provides probability-based predictions and is computationally efficient.

Both methods can handle classification problems, but Decision Trees are more suitable when doctors need simple explanations for individual predictions.

5. Joint final design

The final system will use a cost-sensitive Decision Tree classifier.

The main steps are:

1. Collect patient information such as age, diagnosis, previous admissions and laboratory results.
2. Clean the dataset and handle missing values using imputation.
3. Apply oversampling or cost-sensitive learning to address class imbalance.

4. Train a Decision Tree using the prepared dataset.
5. Adjust the classification threshold according to the 10:1 error cost ratio.
6. Generate understandable risk alerts for doctors.

6. Cost-sensitive decision rule

False Negative cost = 10

False Positive cost = 1

The expected cost of predicting high risk is:

$$Cost(High) = 1 - P \times 0 + (1 - P) \times 1$$

$$= 1 - P$$

The expected cost of predicting low risk is:

$$Cost(Low) = 10P$$

The system should flag a patient as high risk when:

$$1 - P < 10P$$

$$1 < 11P$$

$$\boxed{P > 0.0909}$$

Therefore, the initial theoretical threshold is approximately 9.09%, assuming calibrated probabilities and the stated error costs.

7. Student A – Analytical validation

Consider two illustrative confusion matrices.

First threshold:

- TP = 54
- FN = 6
- FP = 180
- TN = 760

Total cost:

$$Cost = (FN \times 10) + (FP \times 1)$$

$$= (6 \times 10) + 180$$

$$\boxed{Cost = 240}$$

Second threshold:

- TP = 48
- FN = 12
- FP = 70
- TN = 870

$$Cost = (12 \times 10) + 70$$

$$\boxed{Cost = 190}$$

The second threshold has a lower total cost in this example.

Accuracy is not an appropriate selection measure because the dataset contains only 6% readmitted patients. A classifier can achieve high accuracy while missing most high-risk patients.

8. Student B – Experimental validation

The system can be implemented using Python or Weka with a public medical readmission dataset.

The following measures should be evaluated:

- Recall: Measures how many readmitted patients are correctly identified.

- Precision: Measures how many flagged patients are actually readmitted.

- ROC-AUC: Measures the model's ability to distinguish between classes.

- Total Cost: Measures the financial or operational impact of false predictions.

Cross-validation should be used to obtain reliable performance estimates.

The results of the model should also be compared with a simple baseline classifier.

9. Handling class imbalance

Oversampling increases the number of minority-class examples, while undersampling reduces majority-class examples.

Cost-sensitive learning assigns a higher penalty to false negatives. This helps the model identify more high-risk patients.

10. Explaining predictions to doctors

The system should display the main reasons behind each alert.

For example:

- Patient has multiple previous admissions.

- Patient has a longer hospital stay.

- Patient has abnormal laboratory results.

- Patient belongs to a high-risk diagnosis group.

Decision tree paths can provide understandable explanations for these alerts.

11. Limitations

The model may produce incorrect predictions if the dataset contains incomplete or inaccurate information. Changes in medical practices may also reduce prediction accuracy.

The system should be regularly evaluated and updated using new patient records.

12. Joint evaluation and conclusion

The classification problem is difficult because of class imbalance, missing values and unequal error costs.

A cost-sensitive Decision Tree is a suitable solution because it provides interpretable predictions and gives greater importance to identifying high-risk patients.

Before choosing a classifier, a data miner should examine dataset size, attribute types, missing values, class distribution, prediction accuracy, computational requirements and business objectives.